

2025 福州大学大学生电子设计竞赛 暨全国大学生电子设计竞赛选拔赛 设计报告

题目： F 题 自动行驶避障小车

队号： No. 48

摘要

本队小车选用 TI 的 M0G3507 作为核心控制器，硬件部分包括核心控制、巡线、电机驱动、测距避障和电源管理等模块。软件部分采用模块化设计，运用 PID 控制算法和编码电机脉冲计数进行巡线行驶和定点停车，并结合超声波测距数据实现障碍物检测与避让，之后使用陀螺仪模块辅助小车遇障之后返航。

在基础部分，小车从 A 点出发，依靠八路灰度传感器识别路线，利用编码电机控制行驶距离，在 15 秒内到达 B 点并停车，同时发出光提示。接着，小车继续从 A 点出发，到 B 点后通过传感器数据控制电机驱动，以两轮差速方式沿着半弧线行驶到 D 点，再次停车并发出光提示，整个过程不超过 30 秒。在行驶过程中，通过加大拐弯半径使小车的投影能准确覆盖路径上的圆弧顶点 C。

对于提高部分，小车在 D 点停车后，通过已知半径与长度计算得出的距离转向直接返回 A 点并停车，同时发出光提示。当小车从 A 点再次出，行驶到 B 点后沿半弧线向 D 点行驶时，我们使用超声波探测前方，在与障碍物距离小于等于 10cm 处能自动停车并发出光提示，然后通过陀螺仪模块所给角度自动规划路径返回 A 点并停车。

目录

一. 小车模式逻辑	4
二. 小车寻线基础部分	4
2.1 从 A 到达 B 并停止闪烁信号灯	4
电机路程获取	5
2.2 从 A 到达 D 经停 B 点都闪烁信号灯	5
三. 小车寻线提高部分	6
3.1 从 A 到 D 达闪烁信号灯后自动返回 A 点	6
3.2 在 CD 途中遇到障碍物停车后返航至 A 点	7
四. 硬件设计与使用说明	7
4.1 电源供给	7
4.2 GPIO 分配	7
4.3 备用主控实物与功能使用	7
五. 操作注意事项	8
5.1 上电顺序	8
5.2 八路灰度模块注意事项	8
5.3 操作顺序	8

一. 小车模式逻辑

选题按钮逻辑

```
while(1)
{
    uint32_t current_time = sys_time_ms;
    uint32_t stateStartTime = 0;

    if(DL_GPIO_readPins(Switch1_PORT, Switch1_PIN_B19_PIN) == 0)
    {
        Q1 = true;
        Q2 = false;
        Q3 = false;
        Q4 = false;
    }
    if(DL_GPIO_readPins(Switch2_PORT, Switch2_PIN_A21_PIN) == 0)
    {
        Q1 = false;
        Q2 = true;
        Q3 = false;
        Q4 = false;
    }
    if(DL_GPIO_readPins(Switch3_PORT, Switch3_PIN_A22_PIN) == 0)
    {
        Q1 = false;
        Q2 = false;
        Q3 = true;
        Q4 = false;
    }
    if(DL_GPIO_readPins(Switch4_PORT, Switch4_PIN_B20_PIN) == 0)
    {
        Q1 = false;
        Q2 = false;
        Q3 = false;
        Q4 = true;
    }
}
```

二. 小车寻线基础部分

2.1 从 A 到达 B 并停止闪烁信号灯

按下按键一，我们在车辆向前行驶时使用八路灰度传感器与 PID 控制，通过 L520 编码电机获取路程至 80cm 处停车并闪烁 LED 灯。

核心代码：

```
PID
float PID_IR_Calc(int8_t actual_value) {
    static int8_t error_last = 0;
    static float IRTrack_Integral = 0;
    //float dynamic_KP = (abs(actual_value) < 5) ? IRTrack_Trunk_KP * 0.6 :
    IRTrack_Trunk_KP;
    float dynamic_KP;
    if (abs(actual_value) < 10) {
        dynamic_KP = IRTrack_Trunk_KP * 0.8;
    } else if (abs(actual_value) < 20) {
        dynamic_KP = IRTrack_Trunk_KP * 0.9;
    } else {
        dynamic_KP = IRTrack_Trunk_KP;
    }
    int32_t temp_integral = IRTrack_Integral + actual_value;
    IRTrack_Integral = (temp_integral < -100) ? -100 : (temp_integral >
    100) ? 100 : temp_integral;
    int8_t derivative = actual_value - error_last; error_last = actual_value;
    return dynamic_KP * actual_value
    + IRTrack_Trunk_KI * IRTrack_Integral ; + IRTrack_Trunk_KD * derivative;
```

电机路程获取

```
void Motion_Car_Control(int16_t V_x, int16_t V_y, int16_t V_z)
{
    float robot_APB = Motion_Get_APB();

    speed_lr = 0;

    speed_fb = V_x;

    speed_spin = (V_z / 1000.0f) * robot_APB;

    if (V_x == 0 && V_y == 0 && V_z == 0)
    {
        Contrl_Speed(0,0,0,0);

        return;
    }

    speed_L1_setup = speed_fb + speed_spin;

    speed_L2_setup = speed_fb + speed_spin;

    speed_R1_setup = speed_fb - speed_spin;

    speed_R2_setup = speed_fb - speed_spin;

    if (speed_L1_setup > 1000) speed_L1_setup = 1000;

    if (speed_L1_setup < -1000) speed_L1_setup = -1000;

    if (speed_L2_setup > 1000) speed_L2_setup = 1000;

    if (speed_L2_setup < -1000) speed_L2_setup = -1000;

    if (speed_R1_setup > 1000) speed_R1_setup = 1000;

    if (speed_R1_setup < -1000) speed_R1_setup = -1000;

    if (speed_R2_setup > 1000) speed_R2_setup = 1000;

    if (speed_R2_setup < -1000) speed_R2_setup = -1000;

    //printf("%d\t,%d\t,%d\t,%d\r\n",speed_L1_setup,
    speed_L2_setup,speed_R1_setup,speed_R2_setup);
}
```

```
uint32_t current_time = sys_time_ms;

float dt = (current_time - prev_time) / 1000.0f; // ?t(?)

prev_time = current_time;

float delta_distance[4] = {0};

for (int i = 0; i < 4; i++) {

    delta_distance[i] = (Encoder_Now[i] - Encoder_Offset[i])

    / (float)ENCODER_PPR

    * wheel_circumference

    / 1000.0f; // ???

}

float v_left = delta_distance[0];

float v_right = delta_distance[1];

float v_avg = (v_left + v_right) / 2.0f;

total_distance += v_avg * dt;

Contrl_Speed(speed_L1_setup, speed_L2_setup, speed_R1_setup,
speed_R2_setup);

}

void Encoder_Calibration() {

    Contrl_Speed(0, 0, 0, 0);

    delay_us(1000);

    for (int i = 0; i < 4; i++) {

        Encoder_Offset[i] = Encoder_Now[i];

    }

}

float get_total_distance(void) {

    return total_distance;

}
```

2.2 从 A 到达 D 经停 B 点都闪烁信号灯

按下按键二在完成 A 到 B 的直线后，根据黑线出现的转弯情况带来的八路传感器数据改变，调整后双轮转速与转向进行拐弯，并且在 D 点停下。

三. 小车寻线提高部分

3.1 从 A 到 D 达闪烁信号灯后自动返回 A 点

复位后按下按键三，在小车到达 D 点之后，通过陀螺仪模块获取当前姿态角，根据地图数据，转至回 A 点角度，直线行驶，覆盖到 A 点

<pre>// IMU algorithm update void updateIMU(float gx, float gy, float gz, float ax, float ay, float az) { float recipNorm; float s0, s1, s2, s3; float qDot1, qDot2, qDot3, qDot4; float _2q0, _2q1, _2q2, _2q3, _4q0, _4q1, _4q2, _8q1, _8q2, q0q0, q1q1, q2q2, q3q3; // Convert gyroscope degrees/sec to radians/sec gx *= 0.0174533f; gy *= 0.0174533f; gz *= 0.0174533f; // Rate of change of quaternion from gyroscope qDot1 = 0.5f * (-q1 * gx - q2 * gy - q3 * gz); qDot2 = 0.5f * (q0 * gx + q2 * gz - q3 * gy); qDot3 = 0.5f * (q0 * gy - q1 * gz + q3 * gx); qDot4 = 0.5f * (q0 * gz + q1 * gy - q2 * gx); // Compute feedback only if accelerometer measurement valid (avoids NaN in accelerometer normalisation) if(!((ax == 0.0f) && (ay == 0.0f) && (az == 0.0f))) { // Normalise accelerometer measurement recipNorm = invSqrt(ax * ax + ay * ay + az * az); ax *= recipNorm; ay *= recipNorm; az *= recipNorm; // Auxiliary variables to avoid repeated arithmetic _2q0 = 2.0f * q0; _2q1 = 2.0f * q1; _2q2 = 2.0f * q2; _2q3 = 2.0f * q3; _4q0 = 4.0f * q0; _4q1 = 4.0f * q1; _4q2 = 4.0f * q2;</pre>	<pre>q2q2 = q2 * q2; q3q3 = q3 * q3; // Gradient decent algorithm corrective step s0 = _4q0 * q2q2 + _2q2 * ax + _4q0 * q1q1 - _2q1 * ay; s1 = _4q1 * q3q3 - _2q3 * ax + 4.0f * q0q0 * q1 - _2q0 * ay - _4q1 + _8q1 * q1q1 + _8q1 * q2q2 + _4q1 * az; s2 = 4.0f * q0q0 * q2 + _2q0 * ax + _4q2 * q3q3 - _2q3 * ay - _4q2 + _8q2 * q1q1 + _8q2 * q2q2 + _4q2 * az; s3 = 4.0f * q1q1 * q3 - _2q1 * ax + 4.0f * q2q2 * q3 - _2q2 * ay; recipNorm = invSqrt(s0 * s0 + s1 * s1 + s2 * s2 + s3 * s3); // normalise step magnitude s0 *= recipNorm; s1 *= recipNorm; s2 *= recipNorm; s3 *= recipNorm; // Apply feedback step qDot1 -= beta * s0; qDot2 -= beta * s1; qDot3 -= beta * s2; qDot4 -= beta * s3; } // Integrate rate of change of quaternion to yield quaternion q0 += qDot1 * invSampleFreq; q1 += qDot2 * invSampleFreq; q2 += qDot3 * invSampleFreq; q3 += qDot4 * invSampleFreq; // Normalise quaternion recipNorm = invSqrt(q0 * q0 + q1 * q1 + q2 * q2 + q3 * q3); q0 *= recipNorm; q1 *= recipNorm; q2 *= recipNorm; q3 *= recipNorm; anglesComputed = 0; }</pre>
---	--

$_8q1 = 8.0f * q1;$ $_8q2 = 8.0f * q2;$ $q0q0 = q0 * q0;$ $q1q1 = q1 * q1;$	
--	--

3.2 在 CD 途中遇到障碍物停车后返航至 A 点

复位后按下按键四，小车按前述方法从 A 点出发，启用超声波模块，在 CD 段在车与障碍物距离小于等于 10cm 时停止，根据陀螺仪模块的姿态角转向至 A 点方向，前进至 A 点并停车闪烁信号灯。

四．硬件设计与使用说明

4.1 电源供给

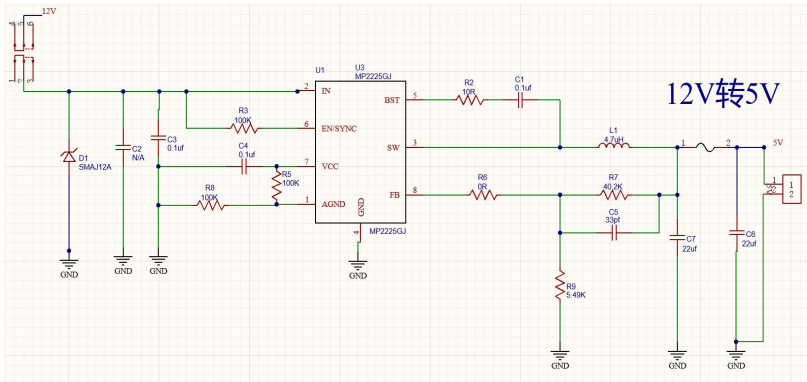


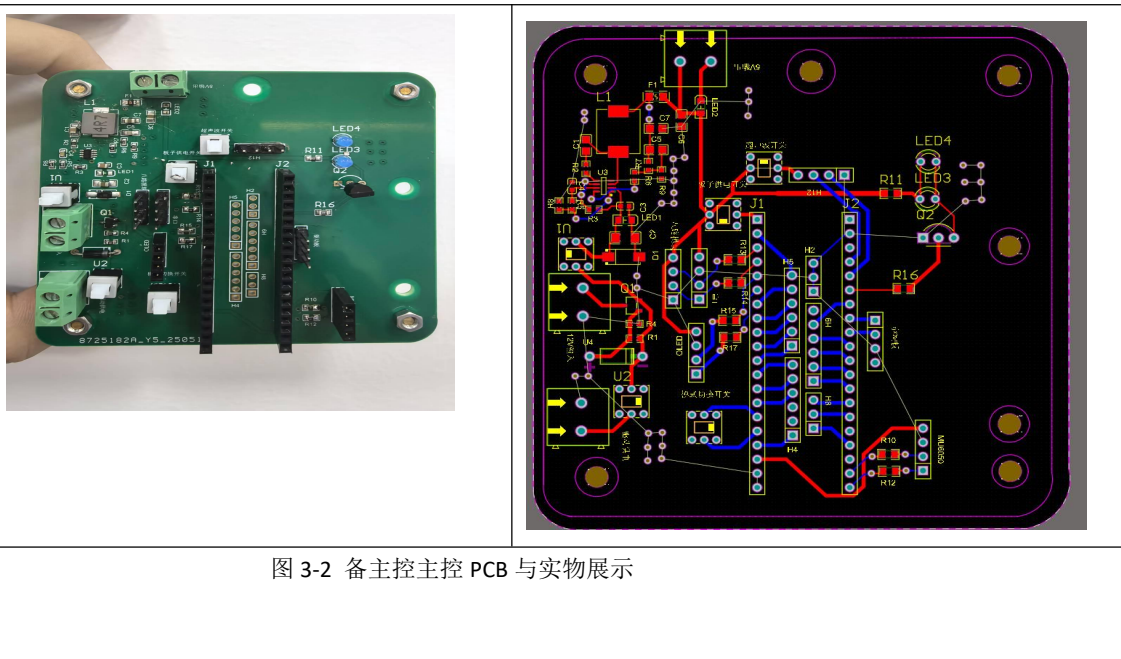
图 4-1 采用 MP2225GJ-Z 芯片设计 12V 转 5V 稳压电路

4.2 GPIO 分配

表 4-1 GPIO 分配表

模块名称	所用引脚		通讯类型
八路灰度传感器模块	A15	A16	I2C
OLED 模块	A17	A18	I2C
驱动板连接串口	B07	B06	串口
陀螺仪模块	A00	A01	I2C
超声波模块	A26	A27	串口

4.3 备用主控实物与功能使用



配备总电源开关放置车辆失控，安装了驱动，超声波开启按钮和模式切换按钮，适应题目需求的功能变换。使用多个端子满足接线需求。

五. 操作注意事项

5.1 上电顺序

需要在驱动板上电后进行其他回路的控制上电，满足驱动板初始化需求。

5.2 八路灰度模块注意事项

上电之后八路灰度传感器需要静止 20s 保证传感器稳定,如若场地进行更换需要对八路灰度传感器重新认色,做法为点按左侧按钮后进行单次认色,多次重复即可。模块本身有断电保存无需每次认色。

5.3 操作顺序

我们将两个部分总四个小题都分配了一个按钮,每题开始前需要按下复位按钮之后按下开始按钮,注意,每次接通电源之后需要等待 4—6s 于程序初始化。(正对车侧由左至右)